



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment-1.1

Student Name: Gagnesh Kakkar

UID: 23BCS11196

Branch: B.E-C.S.E

Section/Group: 23KRG-2B

Semester: 5th

Date of Performance: 15/08/2025

Subject Name: DAA

Subject Code: 23CSH-301

1. Aim: Create Java programs to manage product details, library systems, and student information using classes, inheritance, and abstraction.

2. Objective: To understand stacks.

3. Input/Apparatus Used: Stack are implemented using templates.

4. Procedure:

Step1: Create stack.

Step2: Check underflow and overflow condition.

Step3: Increment top to store element in stack.

Step4: Decrement top after removing element form stack.

Step5: Check is stack empty or not.

5. Code:

```
2
3  class MyStack<T> { 2 usages
4      private int capacity; 2 usages
5      private int top; 6 usages
6      private T[] stack; 4 usages
7
8      public MyStack(int size) { 1 usage
9          capacity = size;
10         top = -1;
11         stack = (T[]) new Object[size];
12     }
13
14     public void push(T element) { 3 usages
15         if (isFull()) {
16             System.out.println("Stack Overflow! Cannot push " + element);
17         } else {
18             stack[++top] = element;
19             System.out.println(element + " pushed into stack.");
20         }
21     }
22
23     public T pop() { 2 usages
24         if (isEmpty()) {
25             System.out.println("Stack Underflow! Cannot pop.");
26             return null;
27         } else {
28             return stack[top--];
29         }
30     }
31
32     public T peek() { 1 usage
33         if (isEmpty()) {
34             System.out.println("Stack is empty. No top element.");
35             return null;
36         } else {
37             return stack[top];
38         }
39     }
40 }
```

```
42
43     public boolean isEmpty() { 3 usages
44         return top == -1;
45     }
46
47     public boolean isFull() { 2 usages
48         return top == capacity - 1;
49     }
50 }
51
52 class StackDemo {
53     public static void main(String[] args) {
54         MyStack<Integer> intStack = new MyStack<>(size: 5);
55
56         intStack.push(element: 10);
57         intStack.push(element: 20);
58         intStack.push(element: 30);
59
60         System.out.println("Top element: " + intStack.peek());
61
62         System.out.println("Popped: " + intStack.pop());
63         System.out.println("Popped: " + intStack.pop());
64
65         System.out.println("Is stack empty? " + intStack.isEmpty());
66         System.out.println("Is stack full? " + intStack.isFull());
67     }
68 }
```

6. Output:

```
10 pushed into stack.
20 pushed into stack.
30 pushed into stack.
Top element: 30
Popped: 30
Popped: 20
Is stack empty? false
Is stack full? false

Process finished with exit code 0
```